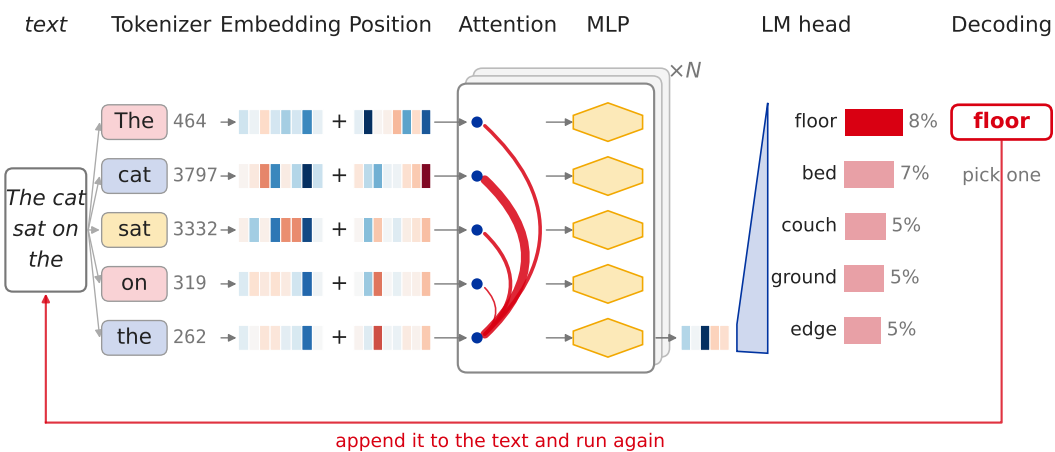
Scaling Laws and Emergence

How big, how much data, and what appears

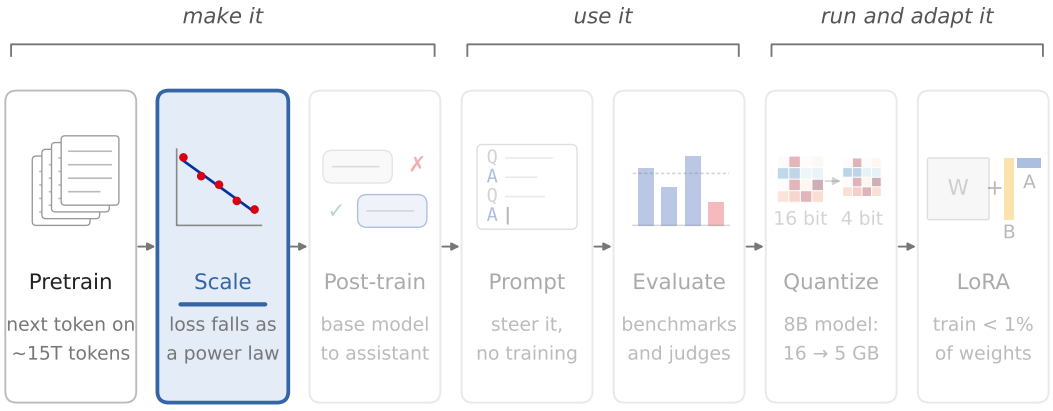
LLM chapter, session 8. Toy numbers unless a slide says otherwise.

Where we are: one forward pass



The pass is built and LLM-7 trained it. Today: how big to make it, and on how much text.

Where we are: the life of a model



Today: the second box. Pretraining at different sizes, and what changes as you scale.

Same budget, two models

DeepMind trained two models with about the same amount of compute:

Gopher (2021)
280 billion parameters
trained on 300 billion tokens

Chinchilla (2022)
70 billion parameters (4× smaller)
trained on 1.4 trillion tokens (4.7× more)

Predict first: on a broad knowledge test (MMLU: multiple-choice questions in 57 subjects, four options each, so guessing gets 25%), which one scores higher?

The small one

	parameters	**tokens**	**MMLU, % correct**
Gopher	280B	300B	60.0%
Chinchilla	70B	1.4T	**67.5%**

Compute check with today's rule $C \approx 6ND$ (in a few frames), in FLOPs (floating-point operations): $6 \times 280\text{B} \times 300\text{B} = 5.0 \times 10^{23}$ against $6 \times 70\text{B} \times 1.4\text{T} = 5.9 \times 10^{23}$. Within 20% of each other.

Gopher was too big for its data. With a fixed budget, a smaller model trained on more tokens wins. Today: how to find the right split, and what happens as the budget grows.

Hoffmann et al. (2022), "Training Compute-Optimal Large Language Models".

Outline

Loss is a power law

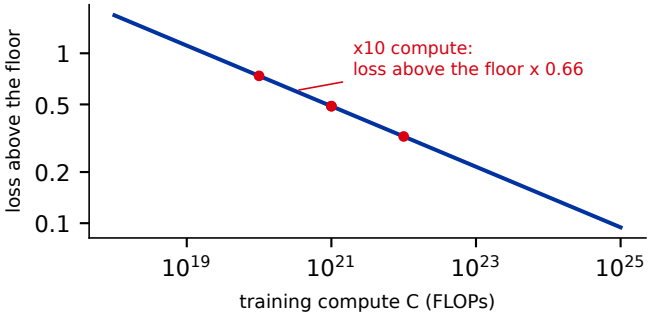
How big, how much data

Emergence

Loss is a power law

Small steps, but predictable ones.

Bigger is better, predictably



Loss never reaches 0: text has its own randomness, a floor ($E = 1.82$ in the fitted law of section 2). Plot what is *above* the floor, at the best model size for each budget.

A straight line: every $10\times$ of compute multiplies it by the same 0.66.

That is a **power law**. Kaplan et al. (2020) found the same shape for model size alone, with data never the limit: exponent 0.076, so doubling N multiplies the loss by $2^{-0.076} = 0.949$, about 5% less. Different quantity, same shape: a fixed factor per doubling, or per $10\times$.

Each step is small, but the line is so straight that a lab can fit it on small runs and predict a run $1{,}000\times$ bigger before paying for it.

The bill: $C \approx 6ND$

Training compute, in floating-point operations:

$$C \approx 6 \cdot N \cdot D \qquad (N \text{ parameters, } D \text{ training tokens})$$

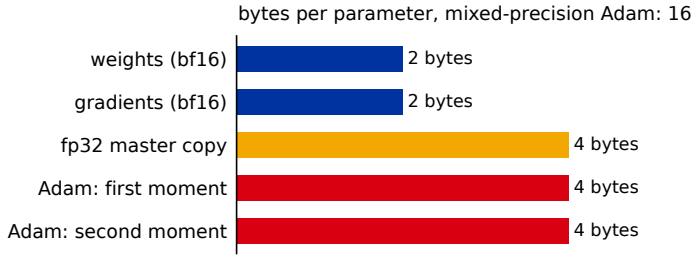
- Forward pass: about 2 FLOPs per parameter per token (one multiply, one add).
- Backward pass: about twice that, 4 more (LLM-5's backprop through every layer).

By hand: Karpathy's GPT-2 run in LLM-7, 124M parameters on 33B tokens:

$$6 \times 124 \times 10^6 \times 33 \times 10^9 \approx 2.5 \times 10^{19} \text{ FLOPs}$$

One budget, two knobs. For the same C you can have a bigger model on less text or a smaller one on more. Gopher and Chinchilla are two such choices.

The other bill: memory



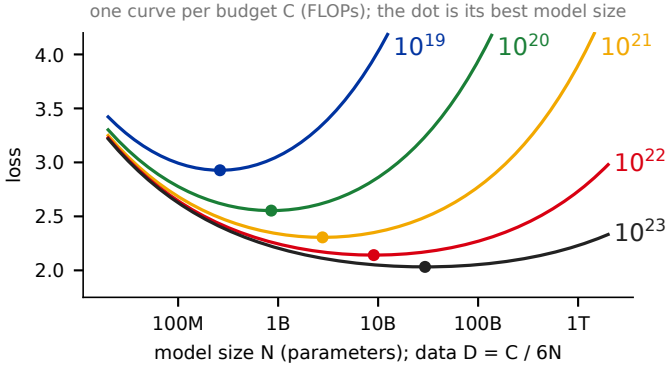
- Training keeps far more than the weights: their gradients, an exact 32-bit copy for the updates, and Adam's two running averages (ch11).
- **By hand:** GPT-2 small, 124M $\times$ 16 bytes $\approx$ **2.0 GB**. An 8B model, 8B $\times$ 16 $=$ **128 GB**: at least two 80 GB GPUs just for this. The activations (LLM-5's intermediate values, kept for backprop) come on top. (bf16 is a 16-bit number format.)

This number comes back in LLM-13: fine-tuning tricks exist mostly to avoid paying it.

How big, how much data

One budget, many ways to spend it.

One budget, many models



Each curve is one budget C. Moving right makes the model bigger, so it sees fewer tokens ($D = C/6N$).

Too small: not enough capacity. Too big: not enough data. The dot is the best split.

Chinchilla's rule from these valleys: grow parameters and tokens **together**, about **20 tokens per parameter**. "For every doubling of model size the number of training tokens should also be doubled" (Hoffmann et al.).

The curves: $L = E + A/N^{\alpha} + B/D^{\beta}$ - the floor, a penalty for a small model, a penalty for little data - with the five constants fitted to hundreds of training runs (Besiroglu et al.'s 2024 refit).

By hand: 10^{21} FLOPs

Two rules, two unknowns:

$$C = 6ND \qquad \text{and} \qquad D = 20N$$

Substitute:

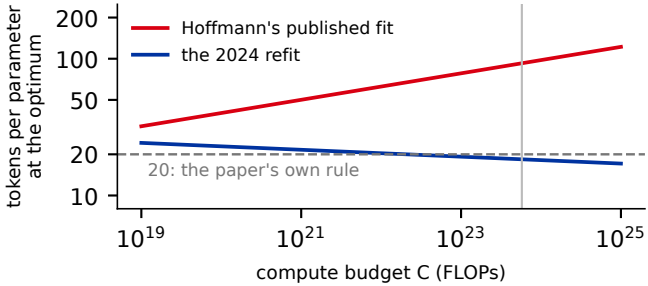
$$C = 6 \cdot N \cdot 20N = 120\,N^2 \quad \Longrightarrow \quad N = \sqrt{C/120} = \sqrt{10^{21}/120} \approx \textbf{2.9 billion}$$

$$D = 20N \approx \textbf{58 billion tokens}$$

Check on Chinchilla itself: $C = 5.9 \times 10^{23}$ gives $N = \sqrt{5.9 \times 10^{23}/120} \approx 70\text{B}$ and $D \approx 1.4\text{T}$ - the model from the cold open.

Ten times the compute buys $\sqrt{10} \approx 3.2\times$ the parameters *and* $3.2\times$ the tokens - not a $10\times$ bigger model.

How to read a claim: the fit



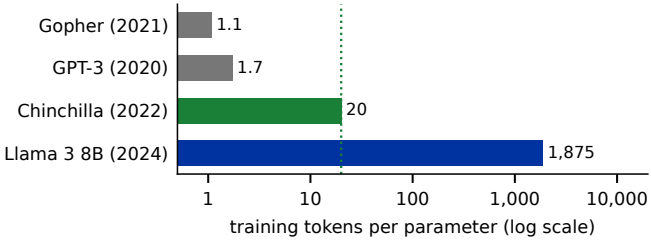
Where the minimum of each fitted law sits, as tokens per parameter. The grey line marks Chinchilla's budget.

The paper's own published constants for $L(N, D)$ put the best split at **50** tokens per parameter at 10^{21} FLOPs, and **93** at Chinchilla's budget - yet Chinchilla itself was trained at 20. A 2024 replication refitted the same data: 22 and 18 at those two budgets, in line with the paper's other two methods. The headline survived; the published fit did not.

Check a fitted law against the claim it is quoted for. Here that meant finding the minimum of each curve on the previous slides for both sets of constants: seconds of compute.

Besiroglu et al. (2024), "Chinchilla Scaling: A replication attempt", arXiv 2404.10102.

Over-training, on purpose



Published sizes: Gopher 280B on 300B tokens, GPT-3 175B on 300B, Chinchilla 70B on 1.4T. Llama 3 8B: over 15T tokens / 8B $\approx$ 1,875 per parameter, 94$\times$ the rule.

- Chinchilla's rule minimises **training** compute. But training is paid once; every answer costs about $2N$ FLOPs per token (the forward share of $6ND$), for every user, for years.
- So labs train small models far past 20 tokens per parameter: a slightly higher loss than the best split for that training compute, but much cheaper to run.

Emergence

Abilities that seem to switch on.

Emergent abilities

- Wei et al. (2022): an ability is **emergent** if it is absent in smaller models and present in larger ones. Plotted against scale, many tasks look like a switch: chance level (no better than guessing), chance level, then suddenly far above it.
- A related trend in GPT-3 (2020): the gap between zero-shot and few-shot (LLM-7) widens with model size; a few examples in the prompt help large models much more than small ones.

If abilities appear suddenly, the smooth loss curves of section 1 cannot predict them. That would make scaling less predictable than it looked. So: is it really sudden?

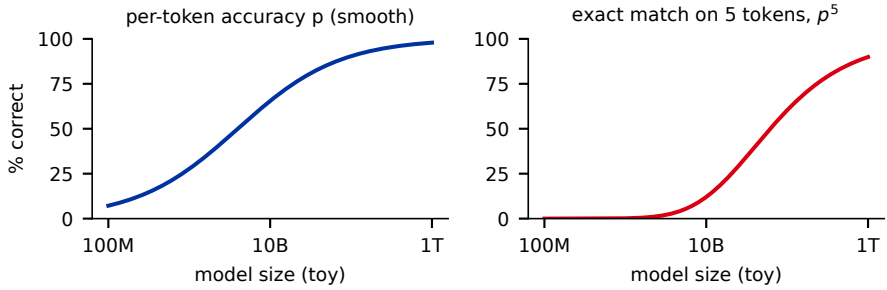
Predict: smooth or sudden?

A toy model family. As the model grows, its **per-token accuracy** p (the chance each answer token is right) rises *smoothly*.

The task: a 5-token answer, scored by **exact match** - all 5 right or nothing.

Predict first: what does the exact-match curve look like as the model grows? Smooth like p, or something else?

The mirage, by hand



Assume each token is right independently, with the same p. Exact match needs all five: p^5. $0.5^5 = 3\%$, $0.7^5 = 17\%$, $0.9^5 = 59\%$, $0.95^5 = 77\%$. Almost nothing, then a "jump", from a model that improved smoothly all along.

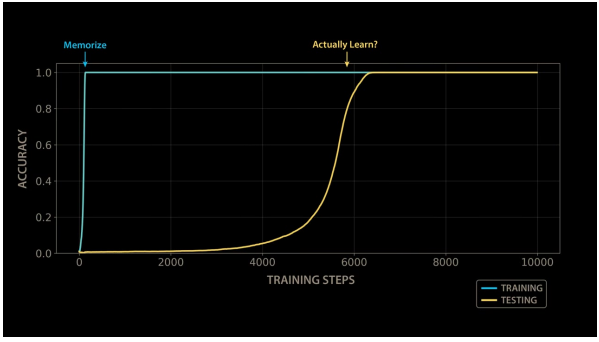
Schaeffer et al. (2023): re-score the same models with a continuous metric (per-token accuracy, edit distance) and many of the published jumps become smooth curves. Those jumps were made by the metric.

Where the debate stands

- The metric explains **many** jumps, not necessarily all of them; the debate is not settled.
- For a user the jump is real anyway: an answer with one wrong digit is a wrong answer. "Smooth underneath" does not help if you need all five tokens.
- What it buys you: with the right metric, a capability can be forecast from smaller models before it "appears".

When you read "ability X emerges at size Y", ask which metric drew the jump.

Another kind of sudden: grokking



Welch Labs, "Grokking" (2025)

Welch Labs' own run: a small transformer learning modular addition from part of all the pairs. Training accuracy hits 100% within about 100 steps: it memorised. Test accuracy stays near 0 until about 3,000, then reaches 100% around 6,000 - some 60× later.

In Power et al.'s (2022) original setup (division mod 97, half the pairs) the gap was about 1,000×.

Sudden in **training time**, not in model size. And the training loss cannot tell memorising from generalising: it was already near 0 long before the test accuracy moved.

A second axis: thinking longer (a preview)

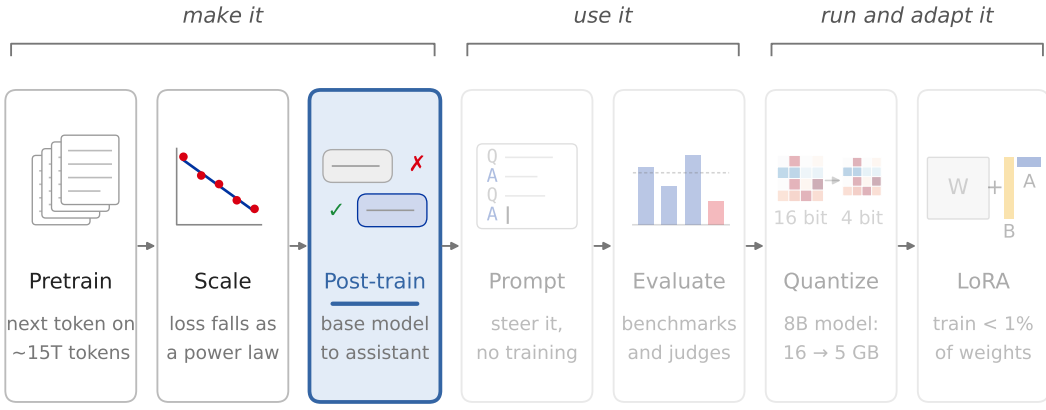
- Everything so far spends compute **before** the model is used: training.
- Reasoning models spend it **while answering**: they write a long chain of thought first, and accuracy improves as they are allowed more of it.
- That is a second scaling axis - test-time compute. How to prompt for it is LLM-10; how such models are trained is the reinforcement-learning chapter.

Two dials: make the model bigger and train it on more text, or let it think for more tokens. They trade off, and both cost compute.

Recap

- **Loss is a power law** in size, data and compute: small, predictable steps toward a floor.
- **The bills**: $C \approx 6ND$ FLOPs to train; about 16 bytes per parameter of memory with Adam.
- **One budget**: grow parameters and tokens together, about 20 tokens per parameter (Chinchilla beat a 4× bigger Gopher). 10^{21} FLOPs $\rightarrow$ 2.9B parameters, 58B tokens. Check fits against their claims.
- **Over-training** small models is deliberate: inference is paid on every token.
- **Emergence**: many jumps come from all-or-nothing metrics (p^5). Grokking is a jump in training time. Test-time compute is the second axis.

Next on the map



Next (LLM-9): from a base model that continues documents to an assistant that answers.

Sources & further reading

- Kaplan et al. (2020), "Scaling Laws for Neural Language Models".
- Hoffmann et al. (2022), "Training Compute-Optimal Large Language Models" (Chinchilla); Besiroglu et al. (2024), "Chinchilla Scaling: A replication attempt".
- Rajbhandari et al. (2020), "ZeRO" (the 16 bytes per parameter).
- Brown et al. (2020), GPT-3; Llama Team (2024), "The Llama 3 Herd of Models".
- Wei et al. (2022), "Emergent Abilities of Large Language Models"; Schaeffer, Miranda & Koyejo (2023), "Are Emergent Abilities of Large Language Models a Mirage?".
- Power et al. (2022), "Grokking"; Nanda et al. (2023), "Progress measures for grokking"; Welch Labs (2025), "Grokking" (video).

Figures: `py_src/scaling_figs.py` (published fits and sizes; the mirage curve is a toy). The grokking still: Welch Labs.